

Department of Computer Engineering

Forensic examination report

The Equifax data breach of 2017, examined with Trivy

Case CSFCL-IA2-2026-005

Course	Cyber Security Forensics and Compliance Laws (CSFCL), IA2
Tool	Trivy 0.74.0 (Aqua Security)
Examination date	28 September 2026
Examiners	Yashasvi Gupta (16010123341) Shubhpreet Kaur (16010123328) Shweta Karandikar (16010123329)

Contents

Case details	4
1. The case: the Equifax breach of 2017	4
1.1 Who Equifax is	4
1.2 How the attack unfolded	4
1.3 Why it was not stopped.....	5
1.4 What was taken	6
1.5 The question this report asks.....	6
2. Methodology	6
2.1 Tool and evidence	6
2.2 The examination.....	7
2.3 Evidence log and chain of custody	8
2.4 Evidence and assumptions	8
3. Findings: the thirteen activities	9
Activity 1: evidence identification.....	9
Activity 2: hash verification.....	9
Activity 3: creating the case.....	10
Activity 4: file system examination	10
Activity 5: user accounts.....	11
Activity 6: deleted files	12
Activity 7: metadata examination	12
Activity 8: browser activity	13
Activity 9: keyword search	13
Activity 10: file signatures and configuration	14
Activity 11: timeline	14
Activity 12: correlating the evidence	15
Activity 13: evidence tagging and exported hashes.....	15
4. The same facts under Indian law	16
4.1 Why Indian law applies	16
4.2 Law in force now	17
4.3 From 13 May 2027: the DPDP Act.....	17

4.4 Reporting and logging deadlines	17
4.5 Using this evidence in an Indian court.....	17
5. Final findings and conclusion	18
6. Limitations	18
7. Sources.....	19

Case details

Item	Detail
Case number	CSFCL-IA2-2026-005
Course	Cyber Security Forensics and Compliance Laws (CSFCL), IA2
Case study	The Equifax data breach of 2017
Evidence ID	CSFCL-IA2-2026-005-EV01, the reconstructed seven-file application dataset at evidence/acis-portal/
Tool	Trivy 0.74.0 (Aqua Security), with sha256sum, unzip, file and Git
Examination	28 September 2026 at 12:54 IST, results in output/20260928T072443Z
Examiners	Yashasvi Gupta (16010123341), Shubhpreet Kaur (16010123328), Shweta Karandikar (16010123329)
Repository	https://github.com/Yashvi2874/csfcl-ia2-equifax-trivy-forensics

Section 1 tells the case. Section 2 covers how we examined a reconstructed seven-file application dataset. Section 3 works through the thirteen activities on the IA2 activity sheet. Section 4 assesses the same facts under Indian law. The final findings, limitations and sources follow. The full demonstration is in the video presentation.

1. The case: the Equifax breach of 2017

1.1 Who Equifax is

Equifax is one of the three big credit bureaus in the United States, with Experian and TransUnion. Banks send it records of people's loans and repayments, and it sells credit reports and scores back to lenders who are deciding whether to lend. The people in those records never signed up with Equifax and cannot take themselves out. In the US the records hold Social Security numbers, the number that stands in for a person's whole financial identity, along with names, dates of birth and addresses.

1.2 How the attack unfolded

The way in was Apache Struts, a free Java framework that companies use to build web applications. Equifax's dispute portal, ACIS (Automated Consumer Interview System), was built on it. ACIS is where people challenged mistakes in their credit files and uploaded documents to back up the challenge.

On 7 March 2017 the Apache team announced a flaw in Struts, CVE-2017-5638, and released a fixed version the same day. The flaw was in the part that handles file uploads. A web request carries a header called Content-Type that says what kind of data is attached. If that header was malformed, Struts tried to run it as a command instead of reading it as text. So anyone on the internet could put a command in that header and the server would run it, with no account and no password. That is remote code execution, the worst kind of flaw, and it scored 9.8 out of 10 on the standard severity scale.

The next day, 8 March, the US government's cyber team warned Equifax to patch. On 9 March Equifax sent its own alert to about 430 staff and mailing lists, telling them to patch within 48 hours. The list was out of date, and the people who ran the dispute portal never got it. On 15 March a scan of 958 internet-facing servers was meant to catch anything still unpatched. It missed the portal.

The fix was out, the warnings had gone round, and the server stayed vulnerable. On 13 May 2017 attackers used the flaw to get in, 67 days after the fix had been public.

Once inside, they set up about 30 web shells, small hidden scripts that kept them in control. On a shared drive they found a configuration file with a database username and password in plain text. The portal needed three databases to do its job, but nothing stopped it reaching further, and those credentials opened 48 more. Over the next weeks the attackers ran about 9,000 database queries. 265 of them returned personal data, which they compressed and carried out in small pieces that looked like ordinary web traffic.

They went unnoticed for 76 days. The device that was meant to inspect the portal's traffic had a certificate that had expired, so for months it could not read that traffic at all. On 29 July, during routine maintenance, staff renewed the certificate. Inspection came back and the suspicious traffic showed up almost at once. Equifax shut the portal down the next day, 30 July, brought in the security firm Mandiant and told the FBI on 2 August, then announced the breach to the public on 7 September, 40 days after it was first spotted.

Date (2017)	What happened	Source
7 March	Apache announces CVE-2017-5638 and releases the fix	Apache Software Foundation
8 March	US-CERT warns Equifax to patch	House report, p. 29
9 March	Internal alert sent to about 430 people, but the list is out of date and misses the portal team	House report, p. 29
10 March	Someone probes the portal, runs whoami, takes nothing	GAO-18-559, p. 11
15 March	A scan of 958 addresses misses the vulnerable server	House report, p. 30
13 May	Attackers get in and install web shells	GAO-18-559, p. 11
May to July	Plaintext credentials found; 48 more databases reached; about 9,000 queries run, 265 return personal data	House report, pp. 32 to 33
29 July	An expired certificate is renewed, and the attack is spotted at once	House report, p. 34
30 July	The portal is shut down	House report, p. 36
7 September	The breach is announced, initially about 143 million US consumers, later revised to about 147 million	GAO-18-559, p. 24

1.3 Why it was not stopped

Equifax told the US auditors that four factors caused the breach, and named a fifth that helped it (GAO-18-559, pp. 15 to 16).

Factor	What went wrong
Identification	The server was not patched. The alert missed the portal team and two scans missed the server
Detection	An expired certificate left encrypted traffic uninspected for 10 to 19 months
Segmentation	The portal could reach 48 databases it had no need for
Data governance	Database credentials were stored in plain text
Query frequency	Nothing limited queries, so about 9,000 were run

1.4 What was taken

Equifax first announced about 143 million affected US consumers on 7 September 2017 and later revised the total to about 147 million. Equifax's May 2018 statement gives approximate US figures: names and dates of birth 146.6 million each, Social Security numbers 145.5 million, addresses 99 million, driver's licence numbers 17.6 million, payment card numbers 209,000. The same person appears in several categories, so the figures are not added. Dispute documents for about 182,000 consumers were also taken. Outside the US, GAO counted nearly one million people in 2018; the UK regulator later found up to 13.8 million UK records accessed, and about 19,000 Canadians were affected. The company agreed in 2019 to a US settlement of at least 575 million dollars, potentially 700 million dollars, was fined by UK regulators, and spent about 1.5 billion dollars on security. The chief information officer, chief security officer and chief executive all left within weeks.

1.5 The question this report asks

With the server's files in front of us, could the attack have been seen before it happened, and what would an examiner find on the machine?

2. Methodology

2.1 Tool and evidence

The breach came down to one vulnerable software component that nobody had patched. Trivy is a scanner built for exactly that: it reads a system's files without running them, names each software component and version, and matches them against public vulnerability databases. It also searches files for passwords and keys, checks configuration for unsafe settings, and writes a list of the software present. That lets us rebuild a version of the Equifax server and show three of the failures from section 1.3 on real software: the unpatched vulnerable component, the exposed credentials and the unsafe configuration. Trivy reads files, so it does not show the monitoring failure, the missing segmentation or the missing query limits; those come from the case record. Trivy is free and scripted, so anyone can repeat the examination and check the results.

We had no access to Equifax's servers, so we rebuilt the part of the portal that mattered. The evidence is in evidence/acis-portal/.

File	What it stands for
lib/struts2-core-2.5.10.jar	The Apache Struts library at the vulnerable version. This is the real file from Maven Central
pom.xml	The four libraries the application declares, all released before March 2017
config/database.properties	Four database usernames and passwords in plain text
config/aws-credentials	Two sets of cloud access keys
config/acis-backup.pem	An RSA private key
config/.env	A session secret and a GitHub token
Dockerfile	Instructions for building the server, which set root as the execution user

The Struts library is the genuine Apache release. Everything else is invented: every password, key and host name. No real Equifax data appears anywhere. The exact Struts version Equifax ran was never published; we use 2.5.10, the last affected release. Nothing in the evidence is ever run. Trivy only reads the files, the way you can list what is inside a zip archive without running the programs in it.

2.2 The examination

One script, run-investigation.sh, does the whole job: it hashes every evidence file, runs five scans, hashes the evidence again, checks nothing changed, and hashes every file it produced. Each step below is one command, and the video shows them running.

The eleven commands, in order. Each one produces a figure in section 3.

```
bash scripts/run-investigation.sh
cat output/latest/evidence-hashes-before.txt
sha256sum -c output/latest/evidence-hashes-before.txt
python3 scripts/show-cve.py
python3 scripts/show-cve.py CVE-2017-5638
unzip -p evidence/acis-portal/lib/struts2-core-2.5.10.jar \
  META-INF/maven/org.apache.struts/struts2-core/pom.properties
grep -E "\(\secrets\)\$|^(\CRITICAL|HIGH|MEDIUM|LOW): " output/latest/03-secrets.txt
cat scripts/trivy-secret-rules.yaml
grep -E "^(Failures|DS-)" output/latest/04-misconfigurations.txt
grep -E "'(bomFormat|specVersion|purl)'" output/latest/05-sbom-cyclonedx.json
cat output/latest/artefact-hashes.txt
```

Two scans need --offline-scan because Trivy otherwise contacts Maven Central, which rate-limits and can stop the scan. The JAR is read with trivy rootfs, because trivy fs only reads dependency lists such as pom.xml and returns nothing for a compiled .jar. The committed results in output/ were produced in a Linux environment; the screenshots in section 3 were captured on macOS, where sha256sum -c is written shasum -a 256 -c. The script detects the platform, and the computation is the same either way.

```

shwetakarandikar@Shwetars-MacBook-Air csfcl-ia2-equifax-trivy-forensics % bash scripts/run-investigation.sh
...
==> Activity 2, verifying evidence integrity
    PASS - all evidence hashes identical before and after examination

Vulnerabilities (filesystem) : 21 {'CRITICAL': 7, 'HIGH': 10, 'MEDIUM': 4}
Vulnerabilities (manifest)  : 88 {'CRITICAL': 34, 'HIGH': 42, 'MEDIUM': 11, 'LOW': 1}
Secrets exposed             : 11 {'HIGH': 2, 'CRITICAL': 9}
Misconfigurations           : 5  {'MEDIUM': 1, 'HIGH': 1, 'LOW': 1, 'CRITICAL': 2}
SBOM components             : 1

*** PRIMARY FINDING - the Equifax root cause ***
CVE-2017-5638  CRITICAL  CVSS v3 9.8
org.apache.struts:struts2-core 2.5.10 -> fixed in 2.3.32, 2.5.10.1
struts2: RCE when performing file upload based on Jakarta Multipart parser

Plaintext credentials (data governance):
HIGH      .env:3 Application session secret committed to configuration
CRITICAL  database.properties:9 Database password stored in plaintext configuration
CRITICAL  database.properties:11 Database password stored in plaintext configuration
CRITICAL  database.properties:15 Database password stored in plaintext configuration
CRITICAL  database.properties:17 Database password stored in plaintext configuration

Evidence integrity: PASS - all evidence hashes identical before and after examination

Written to output/20260928T072443Z (also copied to output/latest)
  
```

Figure 1. The examination script and its summary: 21 filesystem findings, 88 in the manifest, 11 secrets, 5 misconfigurations, and evidence integrity PASS.

2.3 Evidence log and chain of custody

Date	By	What happened
10 September 2026, 15:17 IST	Examination team	Evidence set assembled and committed to Git
28 September 2026, 12:24 IST	Examination team	pom.xml corrected from log4j-core 2.14.1 and jackson-databind 2.9.8 to 2.7 and 2.8.6, because the first versions did not exist in 2017. SHA-256 changed from 1fa533b14d30ad81 to 8dedb0c9591336a1
28 September 2026, 12:54 IST	Examination team	Examination run with Trivy 0.74.0. Hashes before and after identical

Git records every change to the evidence, and .gitattributes stops Git rewriting line endings, which would otherwise change the hashes across machines. After the final evidence baseline was established, no evidence file was modified during the examination.

2.4 Evidence and assumptions

Statement	Basis
The JAR is struts2-core 2.5.10, a genuine release built on 27 January 2017	Evidence: pom.properties inside the JAR, and the SHA-1 match with Maven Central
The JAR carries CVE-2017-5638	Evidence: Trivy's scan output
The Dockerfile specifies root as the container execution user (execution not tested), and the config files keep credentials in plain text	Evidence: our reconstructed Dockerfile and config files, not Equifax's own files
Equifax ran an affected Struts version	Case record. The exact version was never published, so 2.5.10 is our choice
The alert, probe, intrusion and detection dates	Case record, not our evidence

3. Findings: the thirteen activities

Activity 1: evidence identification

A reconstructed seven-file application dataset at evidence/acis-portal/ (selected files, not a whole-disk image), about 1.6 MB, most of it the 1,594,816-byte Struts library. There is no container file such as E01 or raw dd. The SHA-256 of the evidence hash list is e3833f1be425fb980ab7071aaa3ca109ef6c57979e0a70ef7f569b141da2ce26.

```
shwetakarandikar@Shwetans-MacBook-Air csfcl-ia2-equifax-trivy-forensics % find evidence/acis-portal -type f -printf '%P\n' | sort
Dockerfile
config/.env
config/acis-backup.pem
config/aws-credentials
config/database.properties
lib/struts2-core-2.5.10.jar
pom.xml
```

Figure 2. The seven evidence files, including the hidden .env file.

What type of image is it? Strictly none: this is a reconstructed set of seven application files, not an acquired disk image. It holds current files only, with no unallocated space and so no deleted data. *Why make a forensic image?* To work on a copy while the original stays sealed, its hash proving it was untouched. *Why not analyse the original?* Opening files changes timestamps and can overwrite deleted data, which a court could call contamination.

Activity 2: hash verification

FTK Imager is the sheet's example tool; we used sha256sum, which does the same job for a folder of files. For the Struts library there is an independent expected value: Maven Central publishes a SHA-1 for every file.

Field	Value
Hash algorithm	SHA-1
Expected (Maven Central)	070e02e901924a40097b847cff4066c15d3a684b
Calculated (our copy)	070e02e901924a40097b847cff4066c15d3a684b
Result	Match. The JAR is the genuine Apache release

All 7 files were hashed before and after the examination and every hash matched. Both hash lists are byte-for-byte identical, hashing to e3833f1be425fb980ab7071aaa3ca109ef6c57979e0a70ef7f569b141da2ce26.

File	SHA-256
Dockerfile	187488120a5b2f1ec53a6406ad9c5109cfde74e0df606d51cd98b7279af50d77
config/.env	286edecd5b361431fb3312be0aba9dfa71aab0fbf07ac93589717d47be91dce0
config/acis-backup.pem	b59a33dcce23d8c2a2347e03b17753064239232ee5e2df6fd8eacc89d1c90ce
config/aws-credentials	ac80b7424ec90237014e5177843ebd7d4db52768a5aa35b8ddb048b6678a2617
config/database.properties	d1f5a1320a15b8389f9e316545e5b8c92cb2c74168881ba2c27580a83b5b7062
lib/struts2-core-2.5.10.jar	7389713131d3f85aff34f09d8ef15d14e12a1d940b987aed72f148c1f6c36af2
pom.xml	8dedb0c9591336a1b9b912869664f157f6526fd8d7c9ba1b6b9edb651117f378

```

shwetakarandikar@Shwetars-MacBook-Air csfcl-ia2-equifax-trivy-forensics % cat output/latest/evidence-hashes-before.txt
187488120a5b2f1ec53a6406ad9c5109cfde74e0df606d51cd98b7279af50d77 evidence/acis-portal/Dockerfile
286edecd5b361431fb3312be0aba9dfa71aab0fbf07ac93589717d47be91dce0 evidence/acis-portal/config/.env
b59a33dcce23d8c2a2347e03b17753064230232ee5e2df6fd8eacc89d1c90ce evidence/acis-portal/config/acis-backup.pem
ac80b7424ec90237014e5177843ebd7d4db52768a5aa35b8ddb048b6678a2617 evidence/acis-portal/config/aws-credentials
d1f5a1320a15b8389f9e316545e5b8c92cb2c74168881ba2c27580a83b5b7062 evidence/acis-portal/config/database.properties
7389713131d3f85aff34f09d8ef15d14e12a1d940b987aed72f148c1f6c36af2 evidence/acis-portal/lib/struts2-core-2.5.10.jar
8dedb0c9591336a1b9b912869664f157f6526fd8d7c9ba1b6b9edb651117f378 evidence/acis-portal/pom.xml
  
```

Figure 3. Baseline SHA-256 hashes of the seven evidence files, recorded before the examination.

```

shwetakarandikar@Shwetars-MacBook-Air csfcl-ia2-equifax-trivy-forensics % shasum -a 256 -c output/latest/evidence-hashes-before.txt
evidence/acis-portal/Dockerfile: OK
evidence/acis-portal/config/.env: OK
evidence/acis-portal/config/acis-backup.pem: OK
evidence/acis-portal/config/aws-credentials: OK
evidence/acis-portal/config/database.properties: OK
evidence/acis-portal/lib/struts2-core-2.5.10.jar: OK
evidence/acis-portal/pom.xml: OK
  
```

Figure 4. Integrity check after the examination: all seven files return OK.

What is a hash? A fixed-length fingerprint of a file's contents; a SHA-256 hash is 64 hexadecimal characters, while the SHA-1 used for the Maven Central check is 40. *Why does it matter?* Changing one bit changes about half the hash, so tampering shows. *What does a match mean?* The file is unchanged; it does not say who made it.

Activity 3: creating the case

Trivy has no case screen, so the script builds the case: case CSFCL-IA2-2026-005, evidence CSFCL-IA2-2026-005-EV01, examiners Gupta, Kaur and Karandikar, results in output/20260928T072443Z, and a copy at output/latest. Each run writes a new timestamped folder, so runs never overwrite each other.

Activity 4: file system examination

A reconstructed set of seven application files has no partitions, operating-system folders, user profiles or Recycle Bin. Its data is in lib/ (the Struts library) and config/ (three settings files and a private key).

The trivy rootfs scan found 21 known flaws in struts2-core 2.5.10: 7 critical, 10 high, 4 medium. Trivy's database is from 2026, so most were disclosed after the breach. Only 2 were public before the intrusion on 13 May 2017:

Published	CVE	Severity	Trivy's description
4 July 2016	CVE-2016-4465	Medium	Possible DoS attack when using URLValidator
11 March 2017	CVE-2017-5638	Critical	RCE when performing file upload based on Jakarta Multipart parser

Filtering our recorded findings by publication date leaves 1 critical Struts finding known before the intrusion, CVE-2017-5638, the flaw the attackers used; the other 6 critical findings were published from September 2017 onward. We did not reproduce a 2017 scanner or its database; this is our 2026 scan filtered by date. The trivy fs scan of pom.xml found 88 flaws in the four declared libraries (34 critical); 4 were public before the intrusion, 3 of them critical.

```

shwetakarandikar@Shwet-MacBook-Air csfcl-ia2-equifax-trivy-forensics % python3 scripts/show-cve.py
PUBLISHED CVE SEVERITY FIXED IN PUBLIC BEFORE BREACH?
-----
2016-07-04 CVE-2016-4465 MEDIUM 2.3.29, 2.5.13 yes
2017-03-11 CVE-2017-5638 CRITICAL 2.3.32, 2.5.10.1 yes
2017-07-13 CVE-2017-7672 MEDIUM 2.5.12
2017-07-13 CVE-2017-9787 HIGH 2.3.33, 2.5.12
2017-09-20 CVE-2016-8738 MEDIUM 2.5.13
2017-09-20 CVE-2017-12611 CRITICAL 2.3.34, 2.5.11
2017-09-20 CVE-2017-9804 HIGH 2.3.34, 2.5.13
2018-08-22 CVE-2018-11776 HIGH 2.3.35, 2.5.17
2019-12-05 CVE-2012-1592 HIGH 2.5.22
2020-09-14 CVE-2019-0230 CRITICAL 2.5.22
2020-09-14 CVE-2019-0233 HIGH 2.5.22
2020-12-11 CVE-2020-17530 CRITICAL 2.5.26
2022-04-12 CVE-2021-31805 CRITICAL 2.5.30
2023-06-14 CVE-2023-34149 MEDIUM 2.5.31, 6.1.2.1
2023-06-14 CVE-2023-34396 HIGH 2.5.31, 6.1.2.1
2023-12-05 CVE-2023-41835 HIGH 6.3.0.1, 6.1.2.2, 2.5.32
2023-12-07 CVE-2023-50164 CRITICAL 2.5.33, 6.3.0.2
2024-12-11 CVE-2024-53677 CRITICAL 6.4.0
2025-12-01 CVE-2025-64775 HIGH 6.8.0, 7.1.1
2025-12-10 CVE-2025-66675 HIGH 6.8.0, 7.1.1
2026-01-11 CVE-2025-68493 HIGH 6.1.1
-----
21 findings in total. 2 were public before the intrusion on 2017-05-13.
  
```

Figure 5. The 21 known Struts flaws, sorted by publication date; two predate the intrusion.

```

shwetakarandikar@Shwet-MacBook-Air csfcl-ia2-equifax-trivy-forensics % python3 scripts/show-cve.py CVE-2017-5638
Vulnerability CVE-2017-5638
Severity CRITICAL
CVSS 3.1 (NVD) 9.8 CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Package org.apache.struts:struts2-core
Installed 2.5.10
Fixed in 2.3.32, 2.5.10.1
Published 2017-03-11
Found in lib/struts2-core-2.5.10.jar
Title struts2: RCE when performing file upload based on Jakarta Multipart parser
Reference https://avd.aquasec.com/nvd/cve-2017-5638
  
```

Figure 6. CVE-2017-5638 in detail: critical, CVSS 9.8, installed 2.5.10, fixed in 2.3.32 and 2.5.10.1.

```

shwetakarandikar@Shwet-MacBook-Air csfcl-ia2-equifax-trivy-forensics % head -18 output/latest/02-manifest-vulnerabilities.txt

Report Summary

+-----+-----+-----+
| Target | Type | Vulnerabilities |
+-----+-----+-----+
| pom.xml | pom | 88 |
+-----+-----+-----+

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

pom.xml (pom)
=====
Total: 88 (UNKNOWN: 0, LOW: 1, MEDIUM: 11, HIGH: 42, CRITICAL: 34)
  
```

Figure 7. The dependency scan of pom.xml: 88 findings, 34 critical, across the declared libraries.

Which file system? None recorded; this is a reconstructed set of files, not a disk image. The real ACIS ran on Solaris. *Which OS?* The Dockerfile builds on tomcat:latest, a Linux image. *How many user accounts?* One configured execution account (root) in the Dockerfile, plus four database accounts (Activity 5).

Activity 5: user accounts

Username	Where	What it is
root	Dockerfile line 6 (USER root)	Configured container execution user; host-level privileges were not established
acis_app, acis_ro	database.properties lines 8, 10	The portal's own database accounts
consumer_svc, audit_svc	database.properties lines 14, 16	Accounts for two unrelated databases

Primary user? The Dockerfile sets root as the container's execution account (USER root, line 6), which Trivy flags as check DS-0002. This is a configured service account, not a human user, and no human user can be identified from this evidence. *Evidence?* Line 6 says USER root and nothing changes it. *Where is user data?* config/. The consumer and audit accounts have no business on this host; the real ACIS held credentials the same way, which let the attackers reach unrelated databases.

Activity 6: deleted files

The supplied dataset contains seven selected application files and no unallocated-space image or deleted-file artifacts, so deleted-file recovery was not performed. It would need a physical disk image and a carving tool such as PhotoRec; Trivy does not carve files.

S.No	Filename	Deleted?	Recoverable?
1	None in this dataset	No unallocated-space image was supplied	Not performed; recovery needs a physical image

Why are deleted files sometimes recoverable? Deleting usually frees the space but leaves the contents until overwritten. *Always removed at once?* No; it depends on the storage (SSD TRIM or encryption can prevent recovery). *Forensic value?* They show what was on the system, and the deletion time sits on the timeline.

Activity 7: metadata examination

No photographs were supplied. JAR metadata was examined as an application-artifact metadata exercise; photograph or EXIF analysis was not performed.

Field	Value
Format	ZIP (JAR), made by v2.0 UNIX
Built	Friday 27 January 2017, 10:38:41 CET
Identifies	org.apache.struts, struts2-core, version 2.5.10
SHA-256	7389713131d3f85aff34f09d8ef15d14e12a1d940b987aed72f148c1f6c36af2

```

shwetakarandikar@Shwetars-MacBook-Air csfcl-ia2-equifax-trivy-forensics % unzip -p evidence/acis-portal/lib/struts2-core-2.5.10.jar META-INF/maven/org.apache.struts/struts2-core/pom.properties
#Generated by Maven
#Fri Jan 27 10:38:41 CET 2017
version=2.5.10
groupId=org.apache.struts
artifactId=struts2-core
  
```

Figure 8. Metadata inside the JAR: struts2-core 2.5.10, built 27 January 2017.

The build date is 39 days before the flaw was announced, so this version existed before the flaw was public. *What can metadata give?* Version, build time and origin; here, enough to match the library to its flaws. *Useful here?* Yes. *Can it prove who made a file?* No; metadata can be edited. The Maven Central hash match is the independent check.

Activity 8: browser activity

No browser-history databases, cookies, download records or server access logs were included in the reconstructed dataset. Browser activity therefore could not be assessed. *Which sites were visited?* None can be identified from this dataset. *Suspicious downloads?* No download records are present. *Can browser history establish user activity?* On a workstation, yes: its entries carry times, though they record the browser, not who typed, and need support from other records.

Activity 9: keyword search

Searching the evidence for the sheet's eight keywords, password gave the most: four database passwords in database.properties (lines 9, 11, 15, 17), plus the same password in the Dockerfile. secret and backup matched the session secret, AWS keys and the private key; confidential, customer and salary matched nothing. A keyword only finds words, so we also ran Trivy's secret scanner with two custom rules, which found 11 secrets in 4 files, 9 critical:

Severity	File and line	Rule	What was found
Critical	config/.env:4	github-pat	GitHub Personal Access Token
High	config/.env:3	acis-session-secret	Application session secret committed to configuration
High	config/acis-backup.pem:2	private-key	Asymmetric Private Key
Critical	config/aws-credentials:2	aws-access-key-id	AWS Access Key ID
Critical	config/aws-credentials:3	aws-secret-access-key	AWS Secret Access Key
Critical	config/aws-credentials:7	aws-access-key-id	AWS Access Key ID
Critical	config/aws-credentials:8	aws-secret-access-key	AWS Secret Access Key
Critical	config/database.properties:9	acis-db-password	Database password stored in plaintext configuration
Critical	config/database.properties:11	acis-db-password	Database password stored in plaintext configuration
Critical	config/database.properties:15	acis-db-password	Database password stored in plaintext configuration
Critical	config/database.properties:17	acis-db-password	Database password stored in plaintext configuration

```

shwetakarandikar@Shwet-MacBook-Air csfcl-ia2-equifax-trivy-forensics % grep -E "(secrets\\$|^(CRITICAL|HIGH|MEDIUM|LOW):" "output/latest/03-secrets.txt"
config/.env (secrets)
HIGH: ACIS (acis-session-secret)
CRITICAL: GitHub (github-pat)
config/acis-backup.pem (secrets)
HIGH: AsymmetricPrivateKey (private-key)
config/aws-credentials (secrets)
CRITICAL: AWS (aws-access-key-id)
CRITICAL: AWS (aws-access-key-id)
CRITICAL: AWS (aws-secret-access-key)
CRITICAL: AWS (aws-secret-access-key)
config/database.properties (secrets)
CRITICAL: ACIS (acis-db-password)
CRITICAL: ACIS (acis-db-password)
CRITICAL: ACIS (acis-db-password)
CRITICAL: ACIS (acis-db-password)

```

Figure 9. The secret scan: eleven findings across four files. Reconstructed evidence, synthetic credentials.

Our first rule missed the read-only password on line 11; we found that by hand and widened the pattern. The scanner also missed the two secrets in the Dockerfile, which the keyword and configuration checks caught. *Most significant keyword?* password, which led to the four database passwords, two for databases the portal never needed.

```

shwetakarandikar@Shwetars-MacBook-Air csfel-ia2-equifax-trivy-forensics % cat scripts/trivy-secret-rules.yaml
# Custom Trivy secret rules for the ACIS investigation.
# Trivy's built-in rules know token formats like AWS keys, but not an
# application's own property names, so the plaintext database passwords in
# database.properties need their own rule.
rules:
  - id: acis-db-password
    category: ACIS
    title: Database password stored in plaintext configuration
    severity: CRITICAL
    keywords:
      - password
    # The optional middle segment catches acis.db.readonly.password as well
    # as acis.db.password. The first version of this rule missed it.
    regex: '(?i)(acis|consumer|audit)\.db\.(?:[a-z]+\.)?password\s*=\s*(?P<secret>\S+)'

  - id: acis-session-secret
    category: ACIS
    title: Application session secret committed to configuration
    severity: HIGH
    keywords:
      - SESSION_SECRET
    regex: 'SESSION_SECRET\s*=\s*(?P<secret>\S+)'
  
```

Figure 10. The two custom secret rules we added for the ACIS credentials.

Activity 10: file signatures and configuration

Every file's first bytes matched its extension: the JAR is a ZIP (50 4B 03 04), the .pem an RSA key, pom.xml is XML, the rest text. No file was disguised. aws-credentials and Dockerfile have no extension, yet hold sensitive data; Trivy scans by content, so it read them anyway. *Why change a file's extension?* To slip past tools that judge by extension; a content check still sees the true type from the first bytes.

Trivy's configuration scan of the Dockerfile ran 28 tests: 23 passed and 5 failed results across 4 checks, 2 of the results critical (DS-0031 failed once for each secret in an ENV line).

Severity	Check	Dockerfile line	What was found
Critical	DS-0031	10	Secrets passed via build-args or envs or copied secret files
Critical	DS-0031	9	Secrets passed via build-args or envs or copied secret files
High	DS-0002	6	Image user should not be 'root'
Medium	DS-0001	3	'latest' tag used
Low	DS-0026	whole file	No HEALTHCHECK defined

```

shwetakarandikar@Shwetars-MacBook-Air csfel-ia2-equifax-trivy-forensics % grep -E "(Failures|DS-)" output/latest/04-misconfigurations.txt
Failures: 5 (UNKNOWN: 0, LOW: 1, MEDIUM: 1, HIGH: 1, CRITICAL: 2)
DS-0001 (MEDIUM): Specify a tag in the 'FROM' statement for image 'tomcat'
DS-0002 (HIGH): Last USER command in Dockerfile should not be 'root'
DS-0026 (LOW): Add HEALTHCHECK instruction in your Dockerfile
DS-0031 (CRITICAL): Possible exposure of secret env "AWS_ACCESS_KEY_ID" in ENV
DS-0031 (CRITICAL): Possible exposure of secret env "DB_PASSWORD" in ENV
  
```

Figure 11. The Dockerfile configuration scan: five failed results across four checks.

Activity 11: timeline

Two dates come from our evidence: the JAR was built 27 January 2017 (pom.properties), and the CVE record is dated 11 March 2017 (scan output). The rest come from the case record: flaw announced and fixed 7 March, intrusion 13 May, attack spotted 29 July, portal down 30 July, breach announced 7 September. In order: library built, flaw announced and fixed, intrusion 67 days later, 76 days undetected, certificate renewed, attack spotted, portal down, breach disclosed. *What sequence can be rebuilt?* The library existed before its flaw was

public; the fix was available for 67 days before the intrusion; the attack was found only when a certificate was renewed during maintenance.

Activity 12: correlating the evidence

Evidence A	Evidence B	Evidence C	Together
CVE-2017-5638, CVSS 9.8	USER root in the Dockerfile	Four plaintext database passwords	These findings illustrate a potential attack chain involving vulnerable software, a configured root user and exposed example credentials. Exploitation and database access were not tested.

What follows when several independent artifacts agree? Confidence rises. One finding could be a misconfiguration or a tool error; a vulnerability record, a configuration file and a credentials file all pointing the same way, with consistent dates, would all have to be wrong at once.

Activity 13: evidence tagging and exported hashes

Evidence ID	Artifact	SHA-256 (first 16)	Why
Evidence-01	lib/struts2-core-2.5.10.jar	7389713131d3f85a	Contains CVE-2017-5638
Evidence-02	config/database.properties	d1f5a1320a15b838	Four plaintext passwords
Evidence-03	Dockerfile	187488120a5b2f1e	Runs as root, secrets in ENV
Evidence-04	config/aws-credentials	ac80b7424ec90237	Two cloud key pairs
Evidence-05	config/acis-backup.pem	b59a33dcce23d8c	Unprotected private key
Evidence-06	config/.env	286edecd5b361431	Session secret and token
Evidence-07	pom.xml	8dedb0c9591336a1	Declares four libraries

The SHA-256 hashes of every file the examination exported, so they can be checked later with sha256sum -c:

Output file	SHA-256
01-rootfs-vulnerabilities.json	3199c7f3ed91d6aa4426c7b650f47415ba43e21260f111942b85ae015ac80712
01-rootfs-vulnerabilities.txt	ca01fdbcf9259c7285fb2ca7074f7554b17f0b2c21dcb0391a51a769e145025f
02-manifest-vulnerabilities.json	ab5704eb8169119dee0095e512ceb0254a183b23bf8f46946c678320d47e7aac
02-manifest-vulnerabilities.txt	0eb9877094471bb10da6b20fb44b945901c7fbb422d2e5498a516e975b2520a1
03-secrets.json	16bf0f846e1bd338409e7202daddadf35c388b43e90354661200e2c07f142d56
03-secrets.txt	30702d2fb8bea630784f001e37f3c01c8be50798e64c9bcfa35d24de17305412
04-misconfigurations.json	9025b1b910df9508c6a5779b07ff7180f44bd40ac3f3351091f0b5bd9ea7e6ca
04-misconfigurations.txt	86b0af396369f91b2d889b9447a2c4cb7d2bdece291626bba52d9129122cab40
05-sbom-cyclonedx.json	16fdf7d536da0a2035a3cd6314dcfb0cc9a1963056d22f673e21445ed6db147a
evidence-hashes-after.txt	e3833f1be425fb980ab7071aaa3ca109ef6c57979e0a70ef7f569b141da2ce26
evidence-hashes-before.txt	e3833f1be425fb980ab7071aaa3ca109ef6c57979e0a70ef7f569b141da2ce26

```

shwetakarandikar@Shwetars-MacBook-Air csfcl-ia2-equifax-trivy-forensics % cat output/latest/artefact-hashes.txt
# Artefacts produced by this examination
# Case: CSFCL-IA2-2026-005   Generated (UTC): 20260928T072443Z
# Algorithm: SHA-256

3199c7f3ed91d6aa4426c7b650f47415ba43e21260f111942b85ae015ac80712 01-rootfs-vulnerabilities.json
ca01fdbcf9259c7285fb2ca7074f7554b17f0b2c21dcb0391a51a769e145025f 01-rootfs-vulnerabilities.txt
ab5704eb8169119dee0095e512ceb0254a183b23bf8f46946c678320d47e7aac 02-manifest-vulnerabilities.json
0eb9877094471bb10da6b20fb44b945901c7fbb422d2e5498a516e975b2520a1 02-manifest-vulnerabilities.txt
16bf0f846e1bd338409e7202daddadf35c388b43e90354661200e2c07f142d56 03-secrets.json
30702d2fb8bea630784f001e37f3c01c8be50798e64c9bcfa35d24de17305412 03-secrets.txt
9025b1b910df9508c6a5779b07ff7180f44bd40ac3f3351091f0b5bd9ea7e6ca 04-misconfigurations.json
86b0af396369f91b2d889b9447a2c4cb7d2bdece291626bba52d9129122cab40 04-misconfigurations.txt
16fdf7d536da0a2035a3cd6314dcfb0cc9a1963056d22f673e21445ed6db147a 05-sbom-cyclonedx.json
e3833f1be425fb980ab7071aaa3ca109ef6c57979e0a70ef7f569b141da2ce26 evidence-hashes-after.txt
e3833f1be425fb980ab7071aaa3ca109ef6c57979e0a70ef7f569b141da2ce26 evidence-hashes-before.txt
  
```

Figure 12. SHA-256 hashes of every file the examination exported.

4. The same facts under Indian law

This is an academic comparison, not legal advice, and asks what would follow if the same failures happened at Equifax's Indian company today.

4.1 Why Indian law applies

Equifax runs a credit bureau in India, registered with the Reserve Bank under the Credit Information Companies (Regulation) Act, 2005 (CICRA). IT Act sections 1(2) and 75 also reach acts committed outside India that involve a computer in India.

4.2 Law in force now

Law	What it requires and how the findings measure up
IT Act s.43A and SPDI Rules 2011	Reasonable security practices for sensitive data, which includes passwords. An unpatched CVSS 9.8 flaw and plaintext passwords are hard to call reasonable. Compensation has no upper limit
CERT-In Directions 2022	Report a breach within 6 hours; keep 180 days of logs in India. Discussed in 4.4
CICRA 2005, s.19, s.20, s.23	Credit data kept accurate and secure, under privacy principles; wilful breach can be fined up to Rs 1 crore
IT Act ss.66, 66C and s.43	Sections 66 and 66C are offences covering dishonest unauthorised access and identity theft, punishable by up to three years in prison. Section 43 is civil: it gives the victim compensation and carries no imprisonment
RBI Directions for credit bureaus, 2026	Cybersecurity and technology-risk controls for credit information companies

Section 72A does not fit: it needs a contractor to disclose personal data deliberately, not a theft by outsiders.

4.3 From 13 May 2027: the DPDP Act

The Digital Personal Data Protection Act, 2023 is law, but its main duties start on 13 May 2027, eighteen months after the DPDP Rules were notified on 13 November 2025.

Provision	Requirement	Maximum penalty
s.8(5)	Reasonable security safeguards	Rs 250 crore
s.8(6) and Rule 7	Tell each affected person and the Board without delay, then a detailed report to the Board within 72 hours (an extension may be allowed)	Rs 200 crore

On that date, section 43A of the IT Act is also removed and replaced by this Act.

4.4 Reporting and logging deadlines

Indian law would require (hypothetical)	What Equifax actually did (US, 2017)
CERT-In: covered entities report a cyber incident within 6 hours	Reported to the FBI after 4 days and the public after 40 days; different actions from a CERT-In filing, which did not apply in the US in 2017
CERT-In: keep 180 days of logs	Web server logs kept only 14 days, or 30 days online
From 2027: tell affected people and the Board without delay, detailed report to the Board within 72 hours	Affected people heard after 40 days

4.5 Using this evidence in an Indian court

The examination's output would be electronic records, admissible under section 63 of the Bharatiya Sakshya Adhiniyam, 2023 (in force since 1 July 2024). It requires a certificate with

the record, signed by the person in charge and by an expert, giving the hash value of each record and the algorithm (SHA1, SHA256 or MD5). Our SHA-256 values are what that certificate would carry. SHA-256 hashes support evidence-integrity verification; admissibility requires compliance with the applicable electronic-evidence provisions, including the relevant certificate requirements, and matching hashes alone do not establish admissibility.

5. Final findings and conclusion

Area	Finding
Operating system	Linux (tomcat:latest); the real ACIS ran on Solaris
File system	None recorded; a reconstructed set of files
Primary user	Human user not identified; Dockerfile sets root (line 6)
Important files	Seven, tagged Evidence-01 to Evidence-07
Deleted evidence	Recovery not performed: no unallocated space or deleted-file artifacts in this dataset
Browser activity	Not assessed: no browser artifacts were supplied
Metadata	JAR built 27 January 2017, struts2-core 2.5.10, genuine against Maven Central
Vulnerabilities	21 in the library, 2 public before the intrusion, including CVE-2017-5638
Secrets	11 in 4 files, including four plaintext database passwords
Configuration	5 failed results across 4 checks, 2 critical
Evidence integrity	All 7 files unchanged, checked by SHA-256 before and after

Based on the examination of the reconstructed seven-file application dataset, the evidence indicates that the rebuilt dispute portal contains struts2-core 2.5.10, a genuine Apache release built on 27 January 2017, carrying CVE-2017-5638 at CVSS 9.8. The Dockerfile specifies root as the execution user, and the config files store four database passwords, two cloud key pairs, a private key and a token in plain text. None of the 7 files changed during the examination. Counting only flaws public on 13 May 2017, the library has 2, and 1 of them is critical: CVE-2017-5638, the flaw the attackers used. This shows the flaw was public and identifiable before the intrusion. The evidence does not show whether Equifax scanned for it or would have acted on the result. The full evidence set, scripts and outputs are in the case repository: <https://github.com/Yashvi2874/csfcl-ia2-equifax-trivy-forensics>.

6. Limitations

The evidence is a reconstruction built from published reports, not Equifax's own disk. This dataset has no unallocated space and no browser or access-log artifacts, so Activities 6 and 8 could not be performed on it. Trivy reports only published vulnerabilities, so a clean scan is not proof of safety, and our secret rules find only what their patterns describe. Vulnerability counts change as new flaws are published; the figures here come from the scan on 28 September 2026. The Indian law analysis describes the position in September 2026.

7. Sources

- US Government Accountability Office, GAO-18-559, *Actions Taken by Equifax and Federal Agencies in Response to the 2017 Breach* (2018). <https://www.gao.gov/assets/gao-18-559.pdf>
- US House Committee on Oversight and Government Reform, *The Equifax Data Breach* (December 2018). <https://oversight.house.gov/wp-content/uploads/2018/12/Equifax-Report.pdf>
- NVD, CVE-2017-5638; Apache Struts bulletin S2-045. <https://nvd.nist.gov/vuln/detail/CVE-2017-5638>
- The Information Technology Act, 2000; SPDI Rules, 2011; CERT-In Directions, 28 April 2022. https://www.indiacode.nic.in/bitstream/123456789/13116/1/it_act_2000_updated.pdf
- The Digital Personal Data Protection Act, 2023. <https://www.meity.gov.in/static/uploads/2024/06/2bf1f0e9f04e6fb4f8fef35e82c42aa5.pdf>
- The Digital Personal Data Protection Rules, 2025 (G.S.R. 846(E)), notified 13 November 2025. Ministry of Electronics and Information Technology, Gazette of India.
- Commencement notification G.S.R. 843(E), 13 November 2025, setting a phased commencement of the DPDP Act that includes an eighteen-month phase. Ministry of Electronics and Information Technology, Gazette of India.
- The Credit Information Companies (Regulation) Act, 2005; the Bharatiya Sakshya Adhiniyam, 2023.
- Trivy documentation. <https://trivy.dev/docs/>